Hospital Management System

Course Name: Object-Oriented Programming I Lab
Course Code : CSE 202

Submitted To,

Tanjina Helaly
(Assistant Professor)
University Of Asia Pacific

Submitted By,

Reg. No: Naim Rahman Santo - 24101049
Reg. No: Samiha Hafsa Sneha - 2410103
Reg. No: Noshin Tabassum - 24101035
Reg. No: Abir Hossain - 24101051

Date of Submission: 17/11/2025

Table of Content

1. Objective 3
2. Problem Statement 3
3. Software & Tools Used 4
4. OOP Theory / Concepts Applied 4
5. Code Implementation 4
6. GUI 4
   6.1 GUI Design 4
   6.2 GUI Screenshot from Implemented Project 4
7. Member Contribution 4
8. Conclusion 5
   8.1 Challenges Faced during Project 5
   8.2 Lesson Learned (Both Technical and Management aspects) 5

1. Objective

The objective of this lab project is to design and implement a comprehensive Java application with the following key components:

Object-Oriented Programming Concepts

The application will demonstrate fundamental object-oriented programming principles, including:

- **Classes and Objects:** Structuring the program using reusable blueprints and instances.
- **Inheritance:** Establishing hierarchical relationships to promote code reuse and extensibility.
- **Encapsulation:** Protecting data integrity by restricting direct access to class members.
- **Polymorphism:** Enabling methods to behave differently based on the object's type.
- **Abstraction:** Simplifying complex systems by focusing on essential features while hiding details.

Additional Functional Requirements

In addition to OOP concepts, the project will incorporate:

- **Input/Output Handling:** Managing user inputs and displaying outputs efficiently to ensure smooth program interaction.
- **Exception Handling:** Implementing mechanisms to gracefully detect and manage runtime errors, enhancing program robustness.
- **Graphical User Interface (GUI):** Designing an interactive and user-friendly visual interface to facilitate seamless user interaction with the application.

2. Problem Statement

Design a Java program to manage hospital records using classes and arrays.
The system should keep records of doctors, patients, admin personnel, appointments, and patients' treatment details. It should allow functionalities such as scheduling appointments, viewing the list of doctors, and managing patient treatment records. The

application will support three types of users — Admin, Doctor, and Patient — each having different access and functionalities within the system.

3. Software & Tools Used

- Programming Language: Java (JDK version)
- IDE: Eclipse / IntelliJ / VS Code
- OS: Windows
- Other dependencies - Scene Builder

4. OOP Theory / Concepts Applied

OOP Theory / Concepts Applied

1. **Inheritance**
   Inheritance is used to create a structured hierarchy in the project.
   The Personnel class is the parent class, and Doctor, Patient, and Admin all extend this class.
   This helped me avoid repeating common attributes like name, id, age, and gender.
   Each subclass only adds the features that are unique to its role, making the project cleaner and easier to maintain.

2. **Encapsulation**

All important variables in the project (such as name, age, gender, specialty etc are kept private.
They can be accessed only through **getter** and **setter** methods.
This protects the data from unwanted changes and ensures proper control over how the data is used inside the system. Encapsulation also makes the classes more secure and organized.

3. **Polymorphism**
   The project uses **method overriding**, which is one form of polymorphism.
   For example :

The toString() method is overridden in the Doctor, Patient, and Appointment classes to show their information in their own customized format.

The equals() method is overridden in the Personnel class so that two personnel objects can be compared using their IDs.

This allows the same method names to behave differently depending on which class is being used.

4. Abstraction

Abstraction is implemented through the abstract class `Personnel`.

This class contains the common attributes and methods for all of the system, but it does not represent a real user by itself.

Only specific user types such as Doctor, Patient, and Admin extend it and provide full implementation.

This hides unnecessary details and shows only what is needed to the user.

5. User-Defined Exceptions

Two custom exceptions - InvalidDoctorException and InvalidPatientException - are used in the project.

These exceptions make error handling more meaningful because they show clear messages when an invalid doctor or patient ID is entered.

This is an example of applying OOP to make the system safer and more user-friendly.

6. Packages

The project uses packages (library and hospitalapp) to separate the logic files from the application files.

This improves organization, makes the project easier to navigate, and gives it a professional structure.

7. Constructor Usage

Constructors are used to initialize objects when they are created.

For example:

- Personnel generates a random 4-digit ID in the constructor.
- Appointment sets default status to "Scheduled"

Using constructors ensures that every object starts with correct and complete data.

8. Object Composition

Some classes contain objects of other classes.
For example:

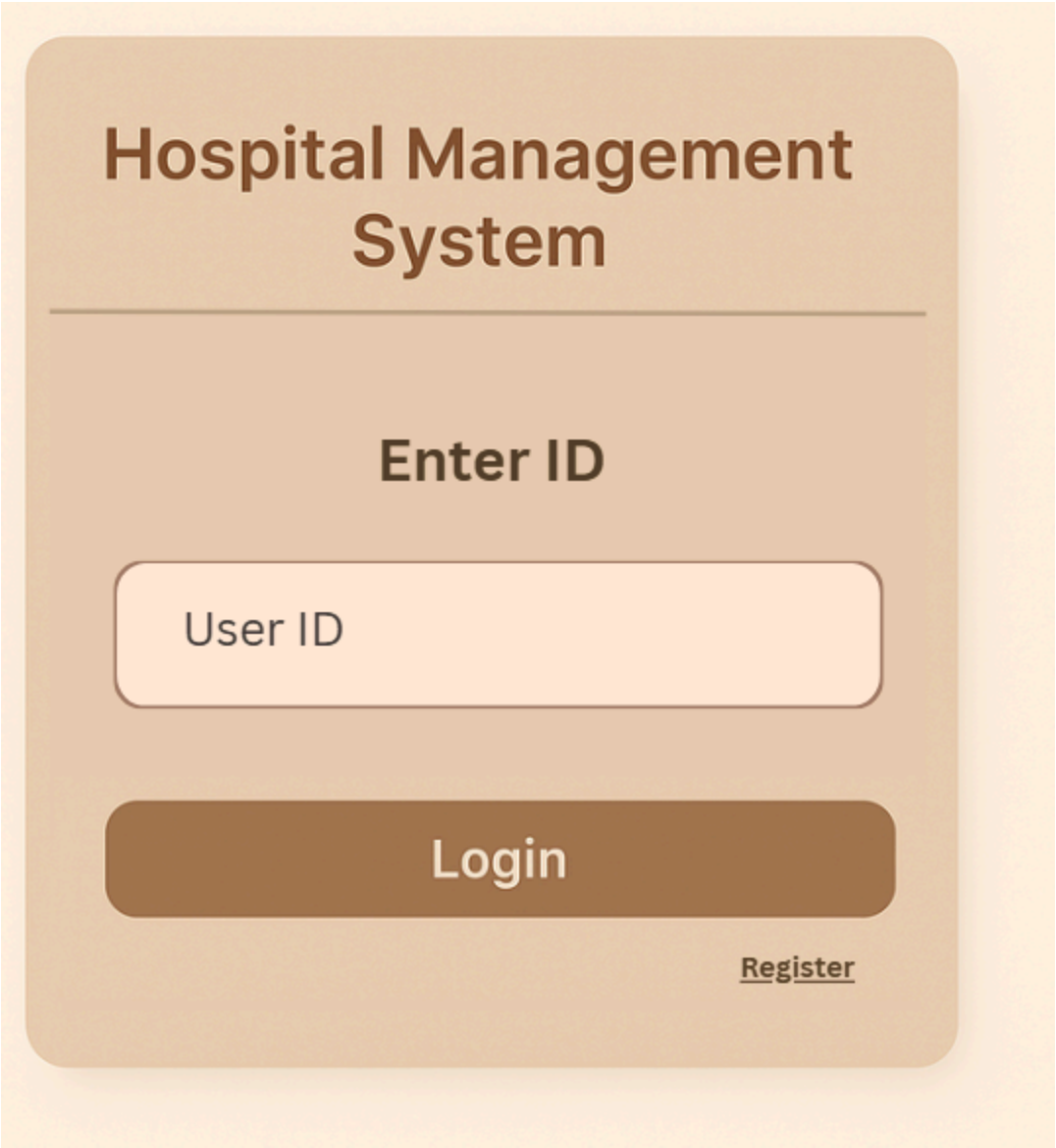
- Appointment contains a Doctor object and a Patient object.
- PatientRecord also contains both doctor and patient objects.

This shows “has-a” relationship, another important OOP concept.

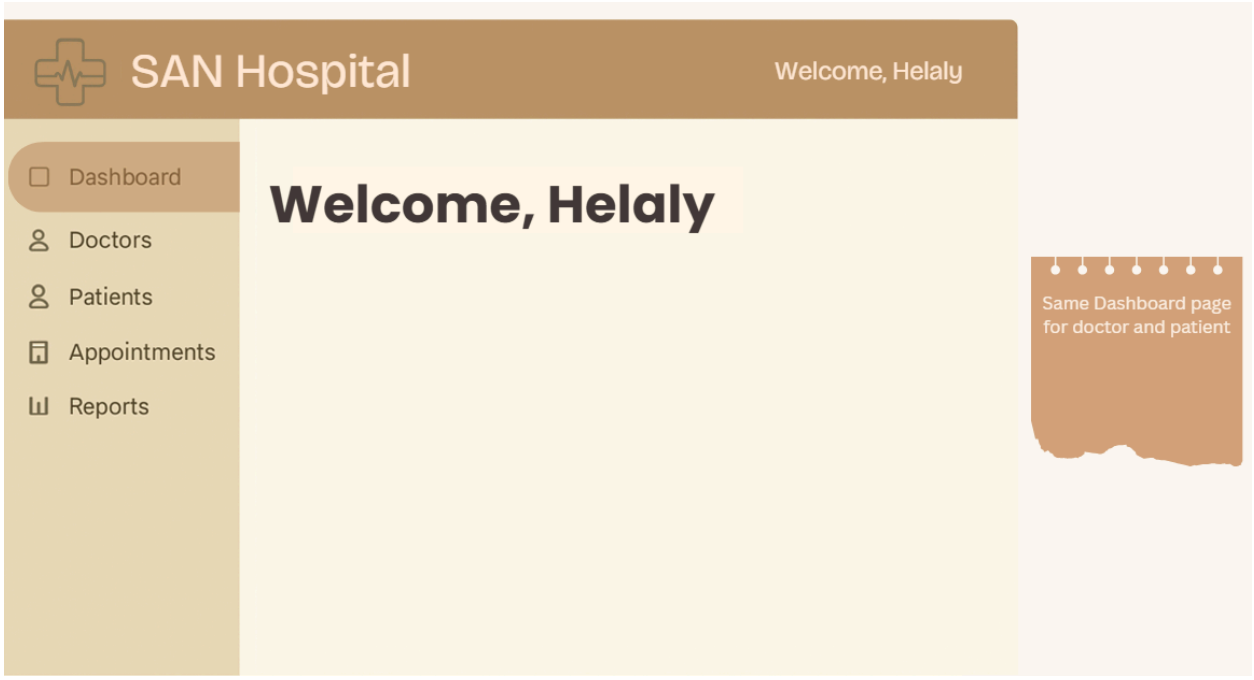
6. GUI

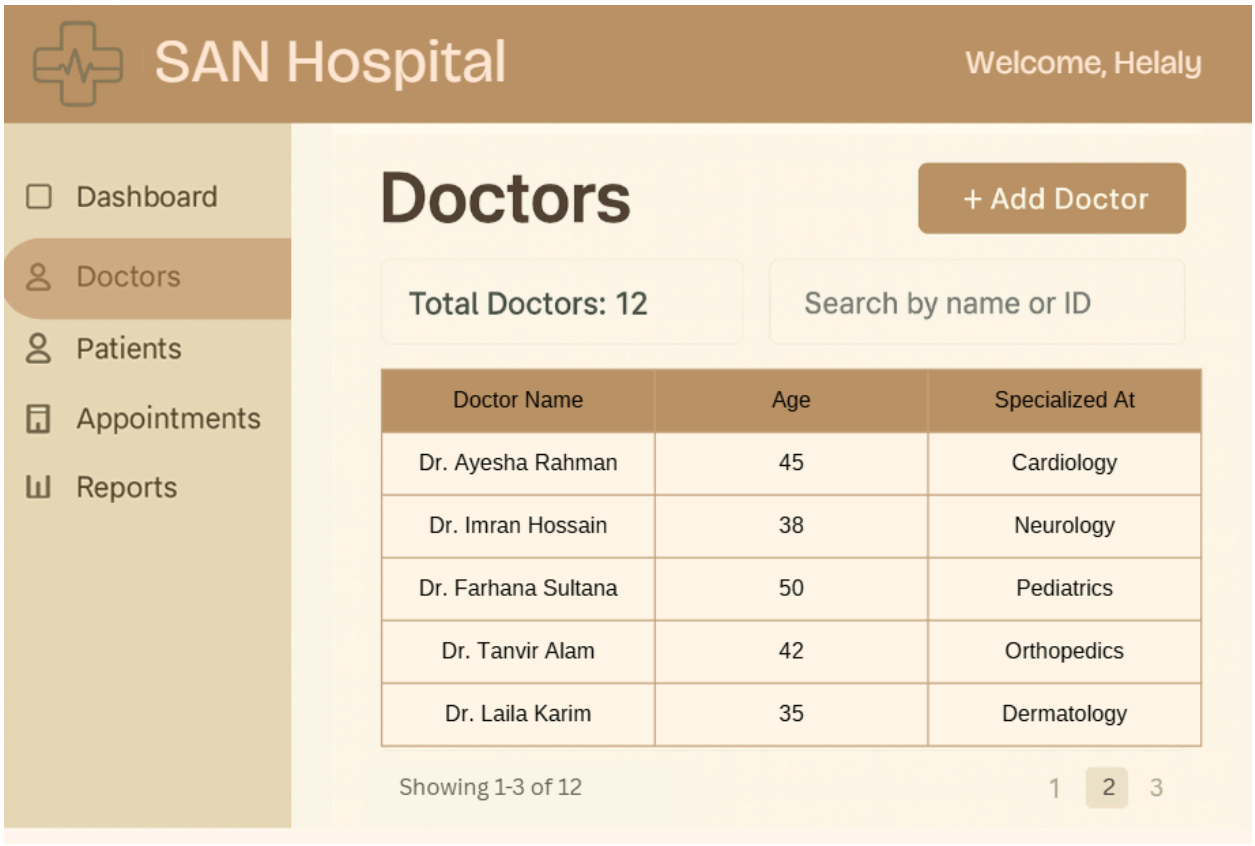
6.1 GUI Design

(Home page)

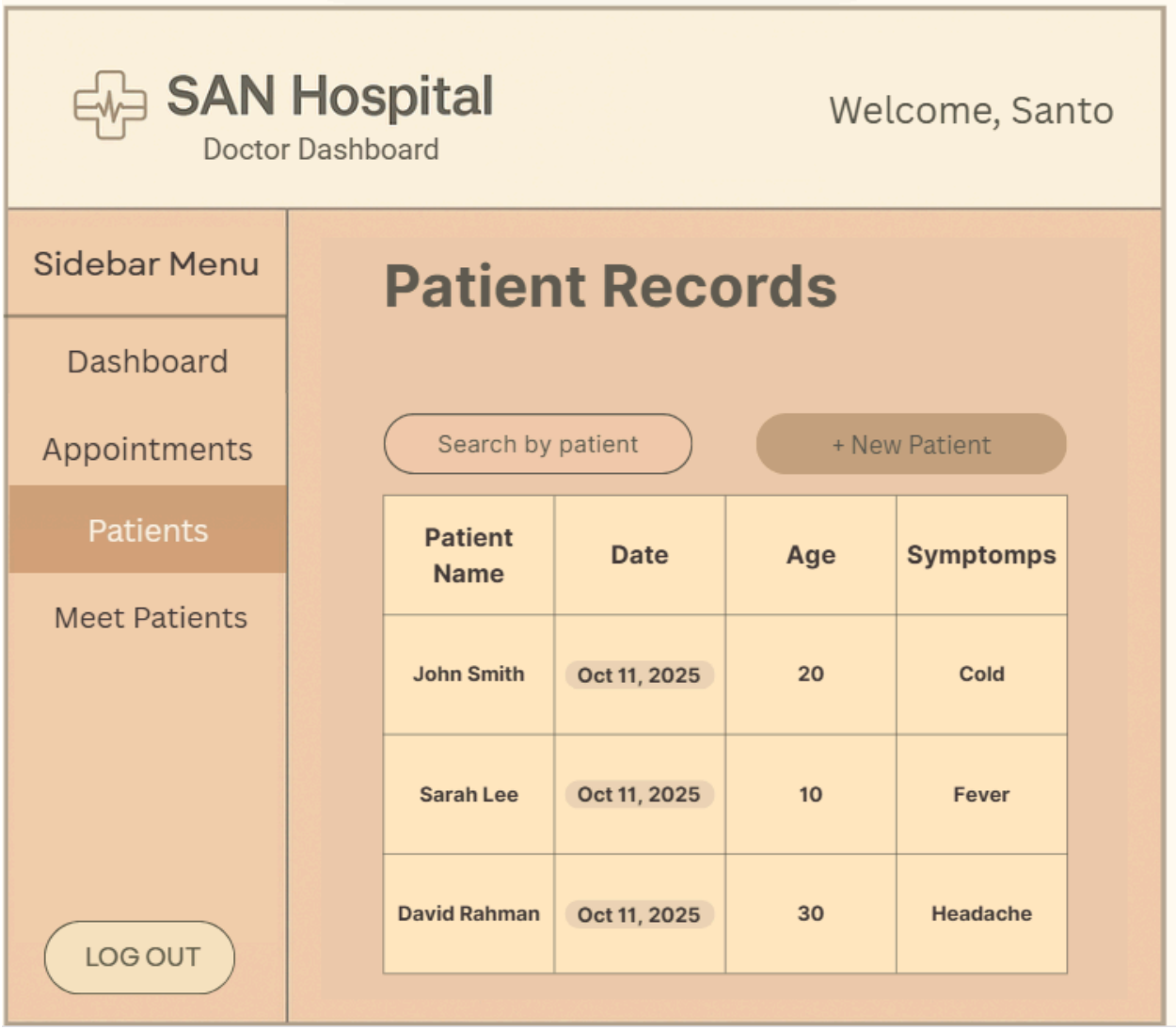


(Login page for Admin, Doctor , Patient)

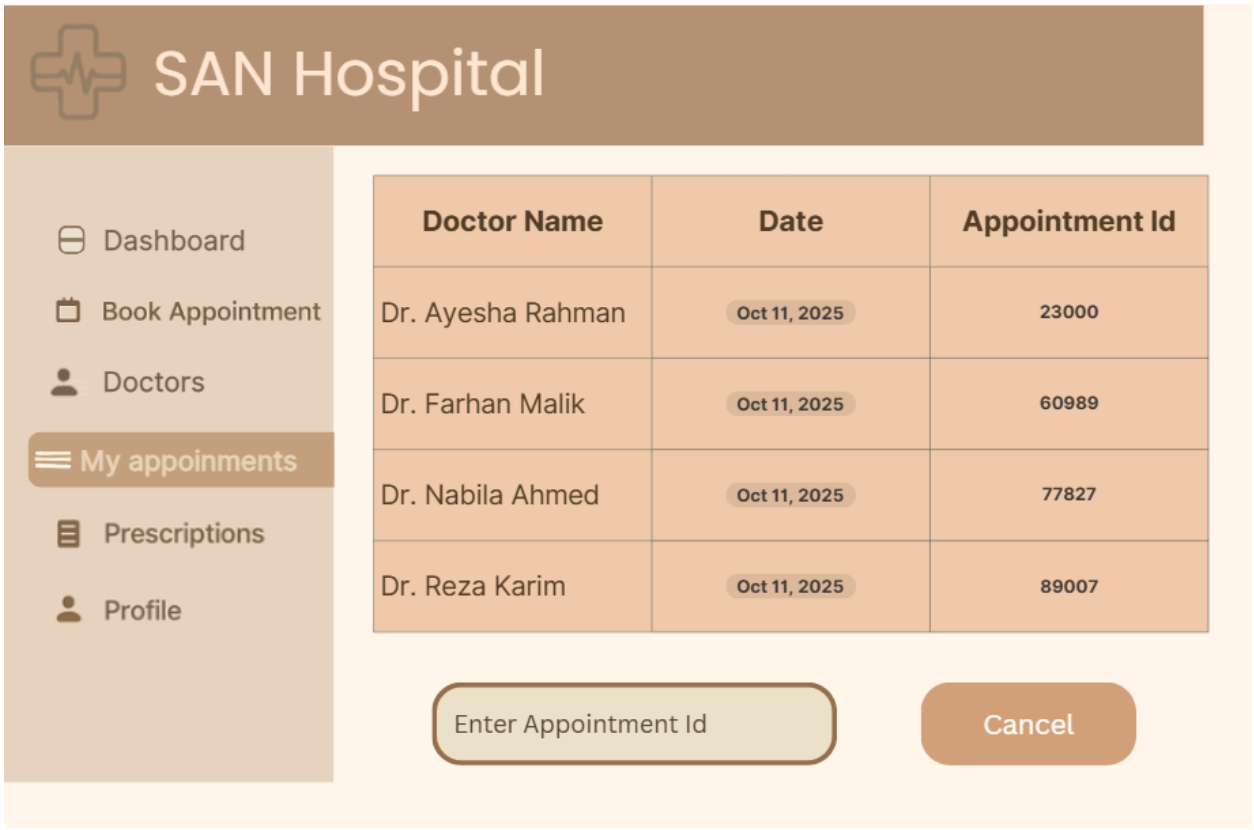




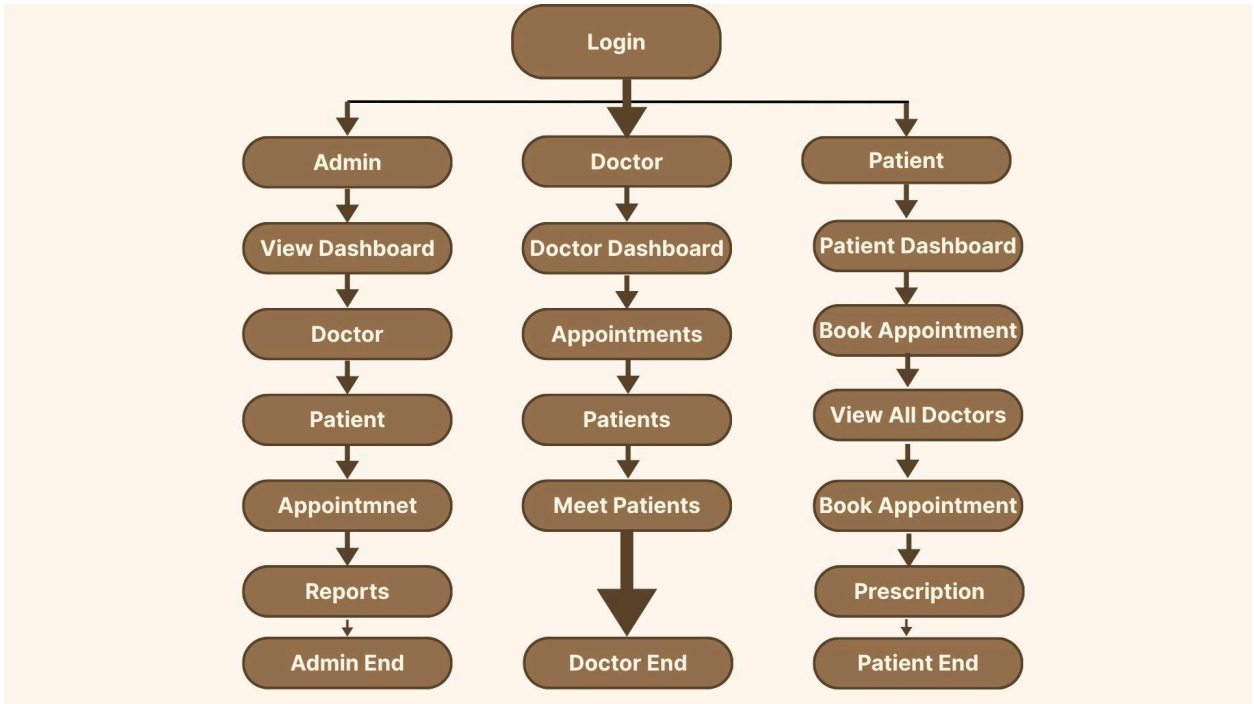
(This is a Admin page. There will be same Add doctor page for other users)



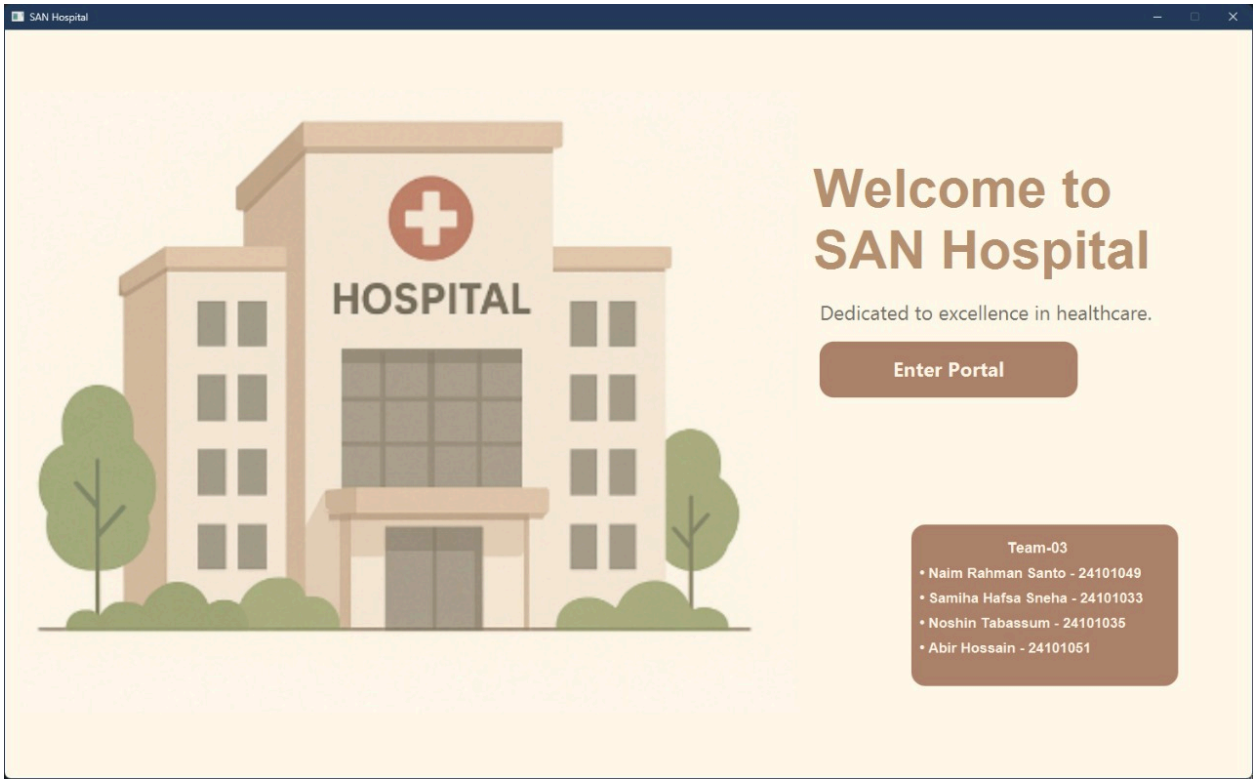
(This is a Doctor page. There will be same Patient list, Appointment list, Report list)



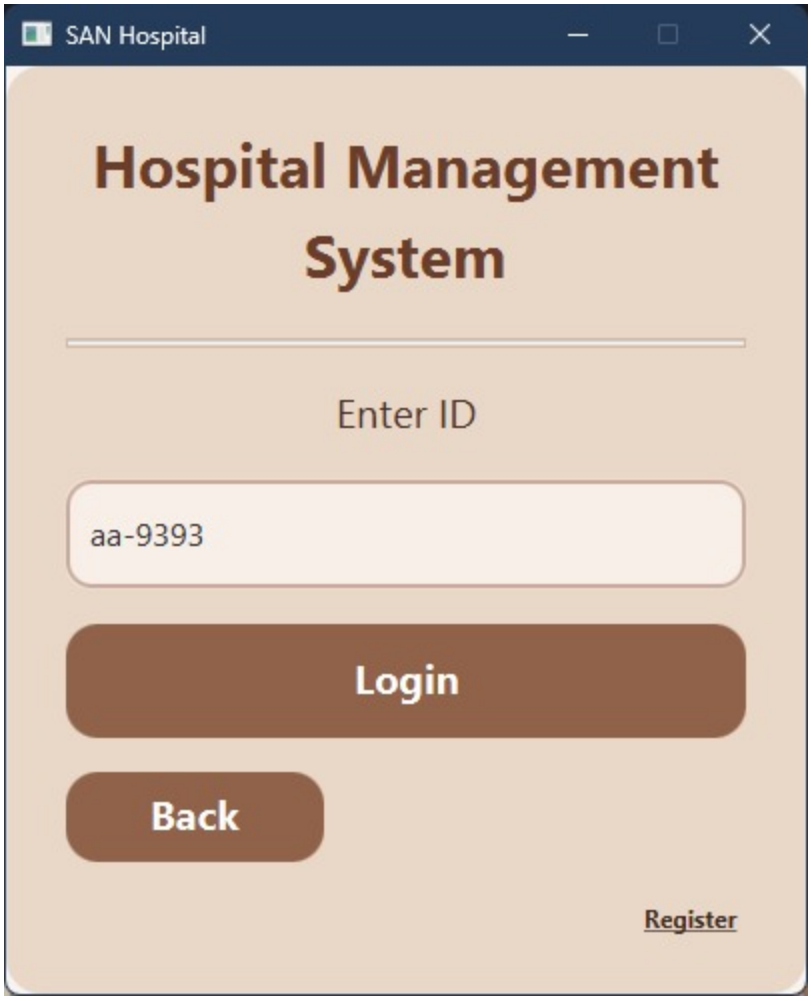
(This is a Patient Page. Patient can see their appointments)



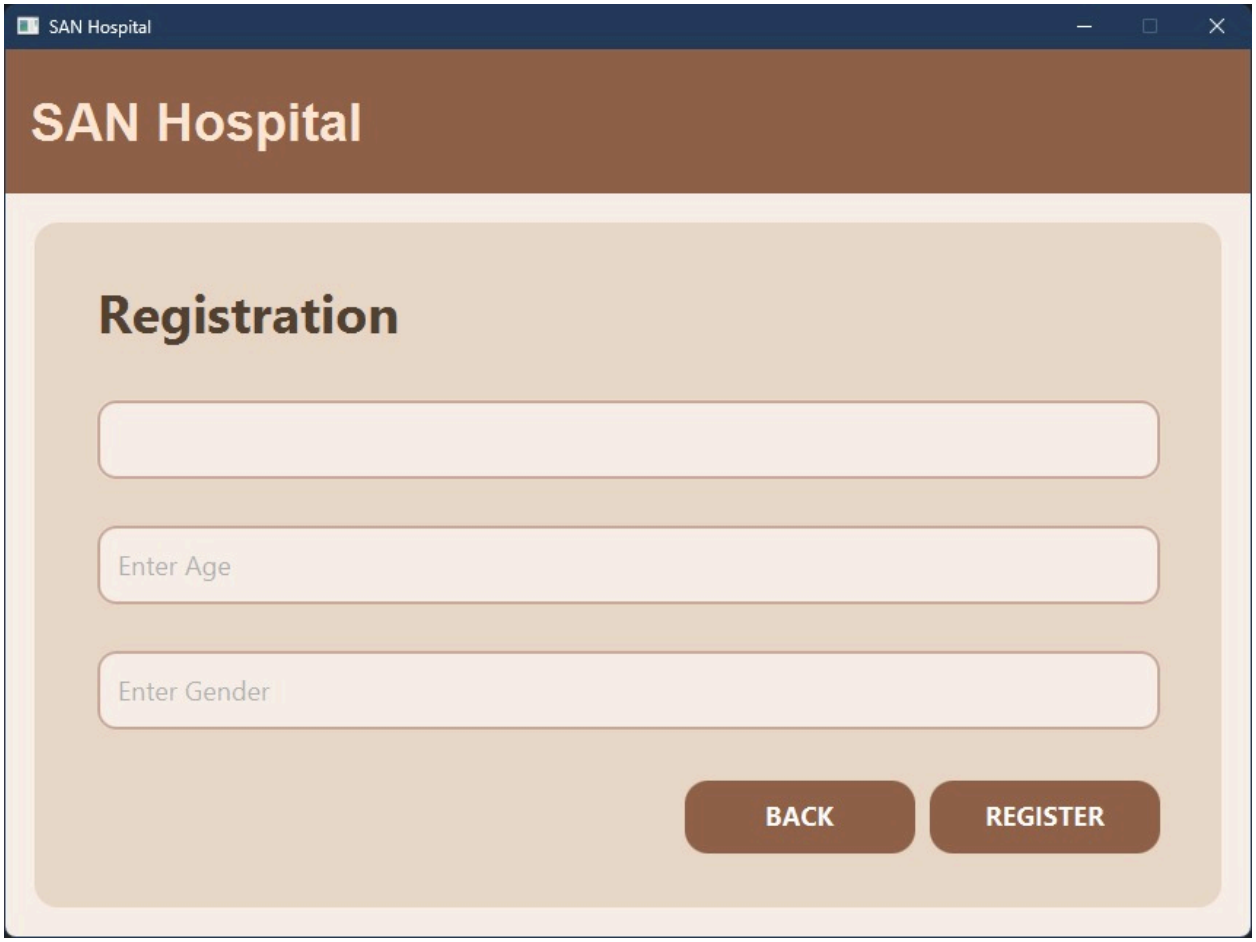
6.2 GUI Screenshot from Implemented Project



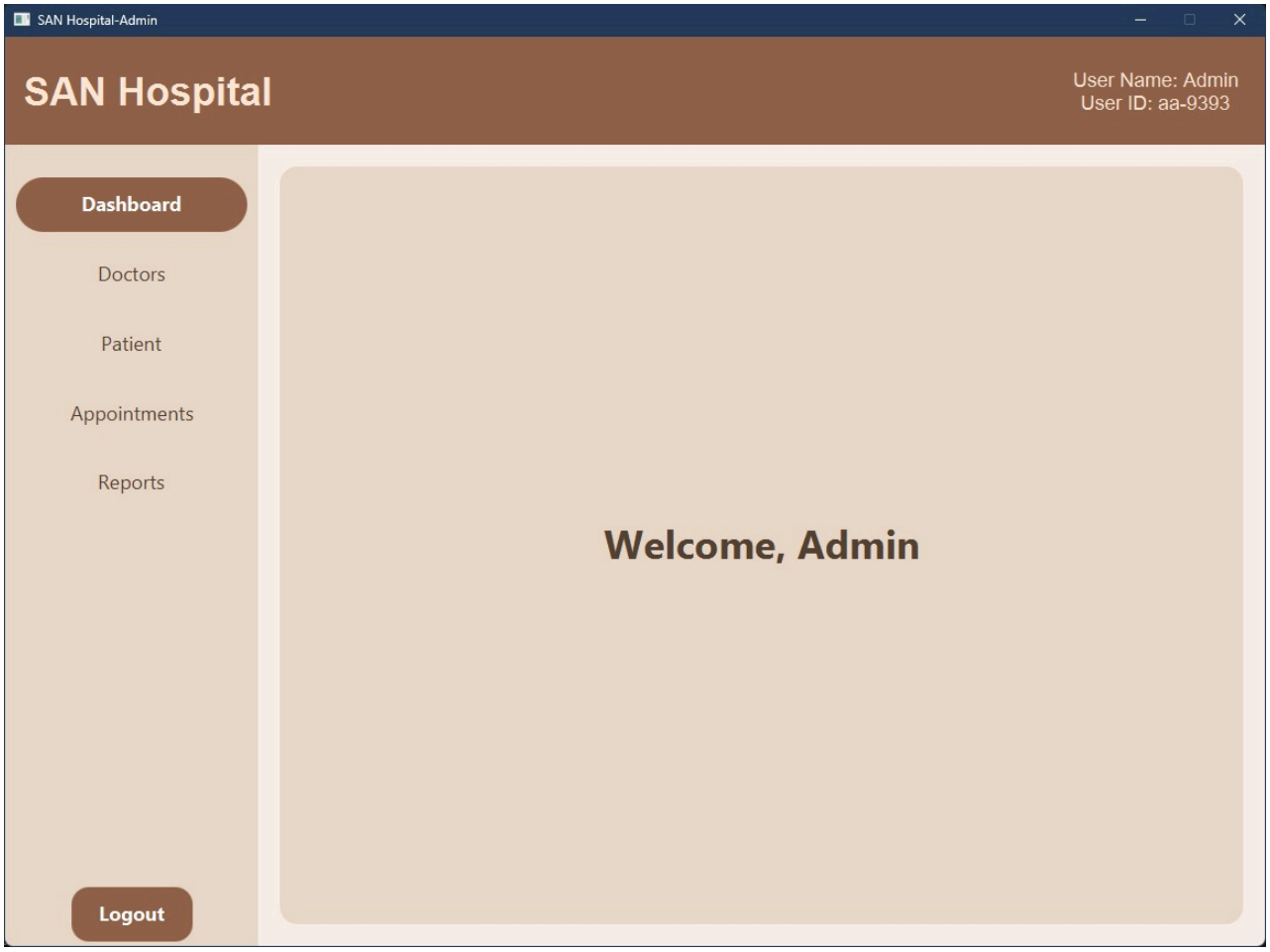
(This is our Home page)



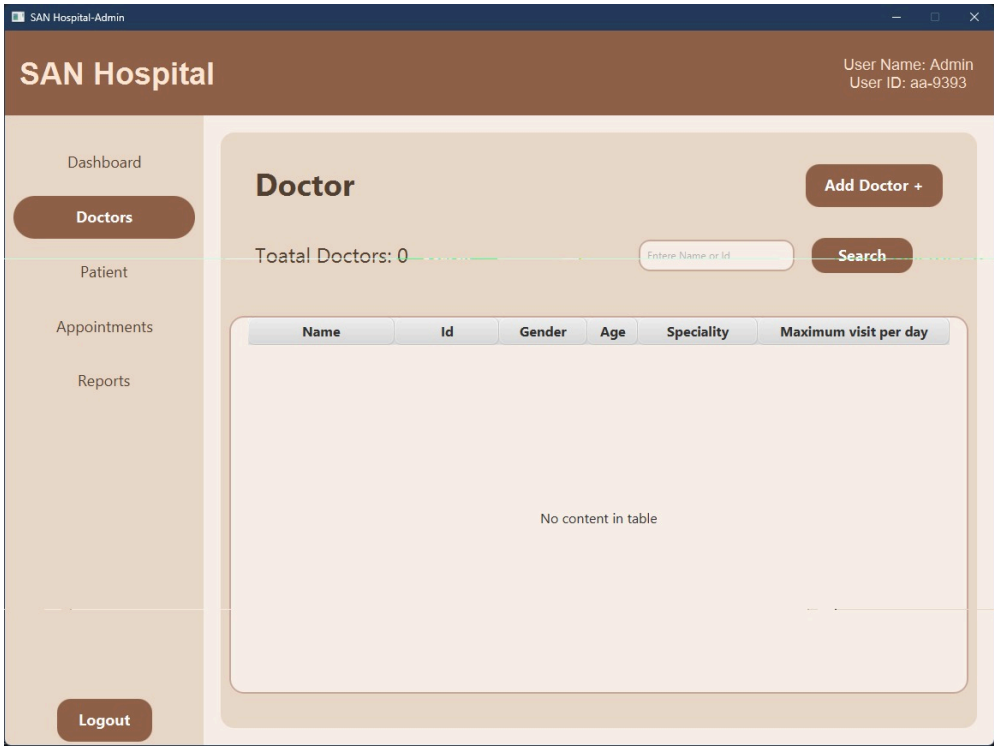
(This is our login page for Admin , Doctor, Patient)



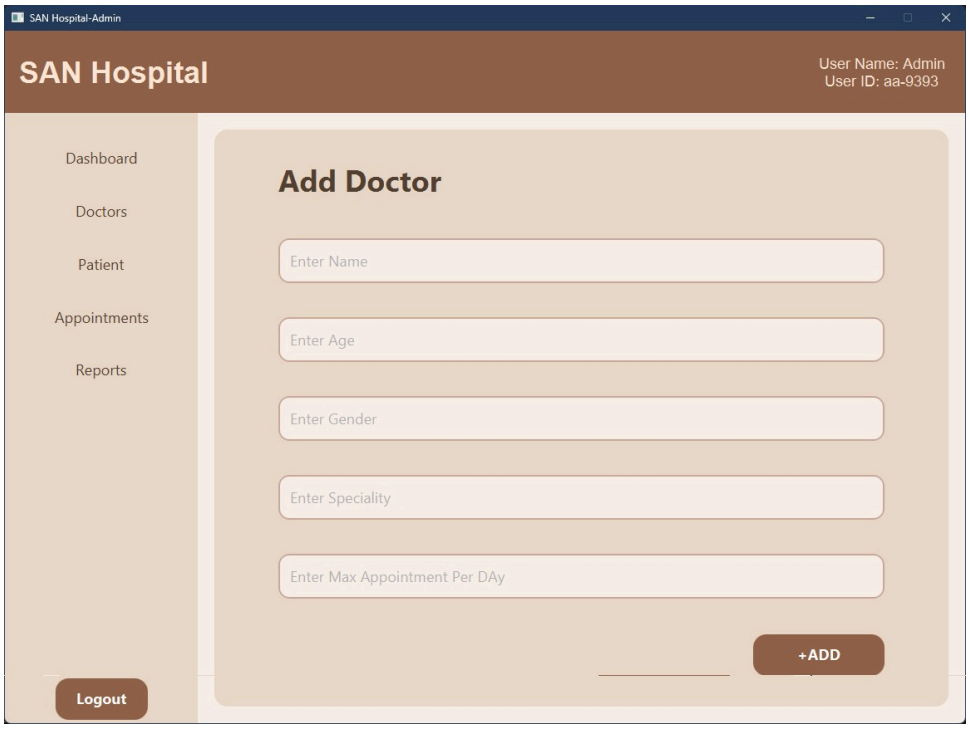
(When a User click Register, this page appears and asks the user to create an account)



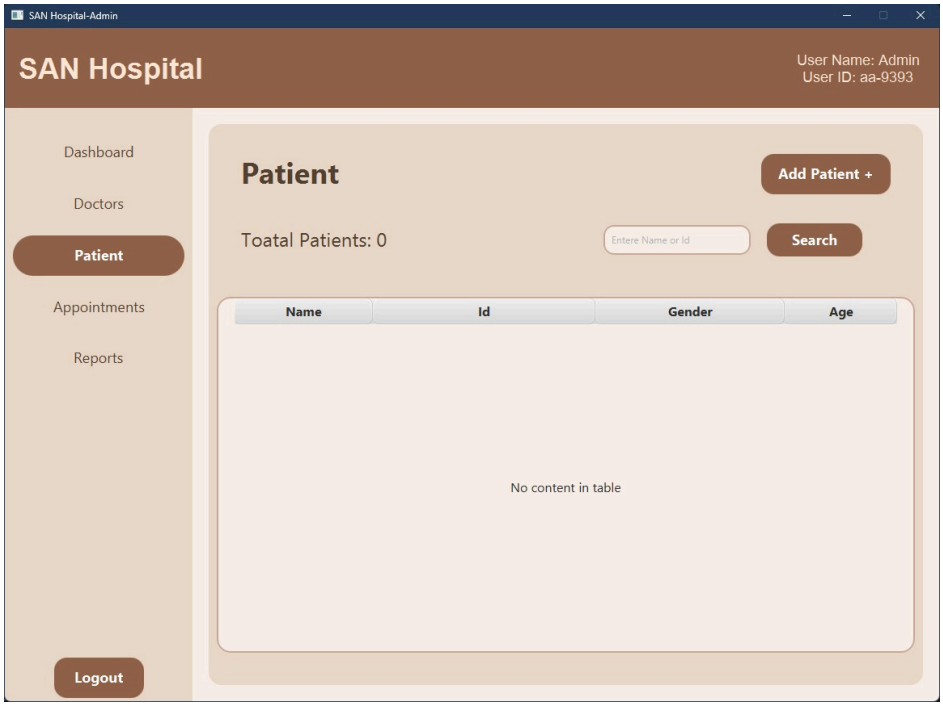
(This is the Dashboard for Admin Page)



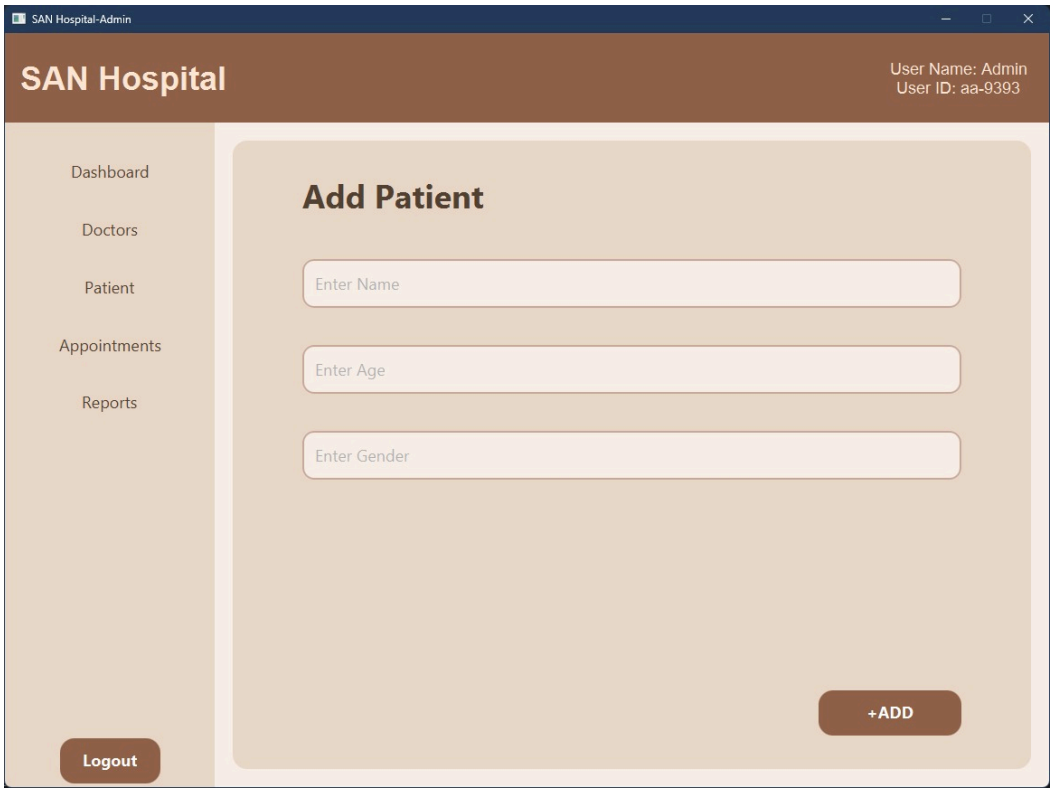
(This is a Doctor list of Admin page where the Admin can see the doctor information)



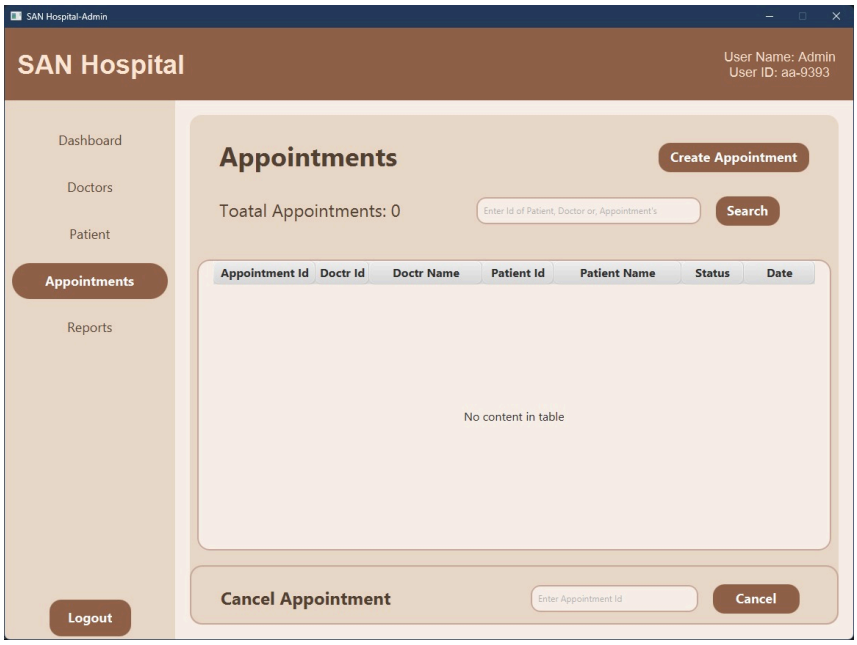
(When Admin click Add Doctor, it tells Admin to add a doctor by entering all the doctor's information)



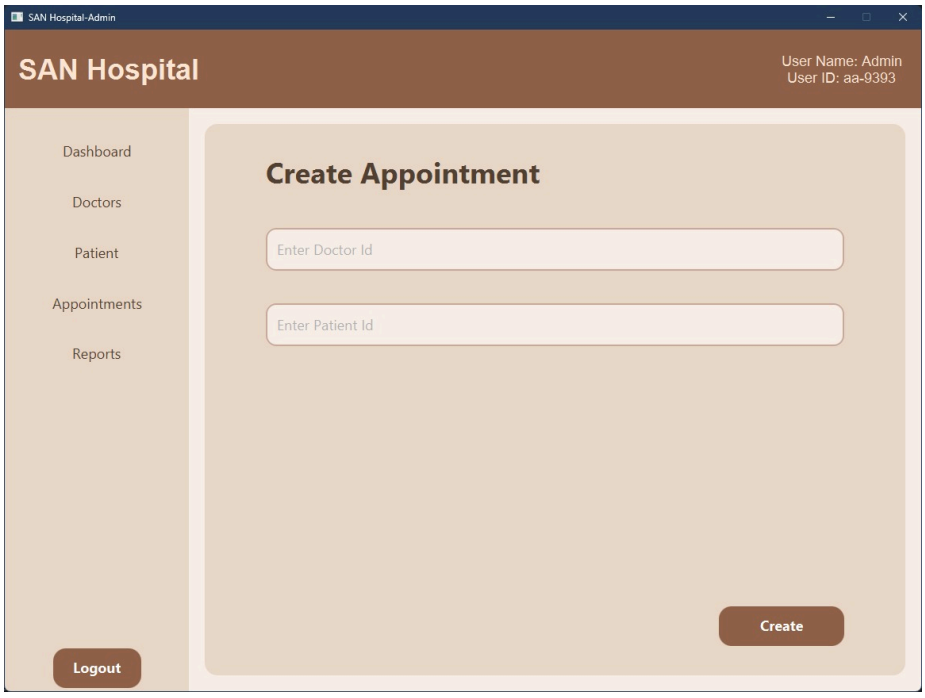
(This is a Patient list of Admin page where the Admin can see the Patient information)



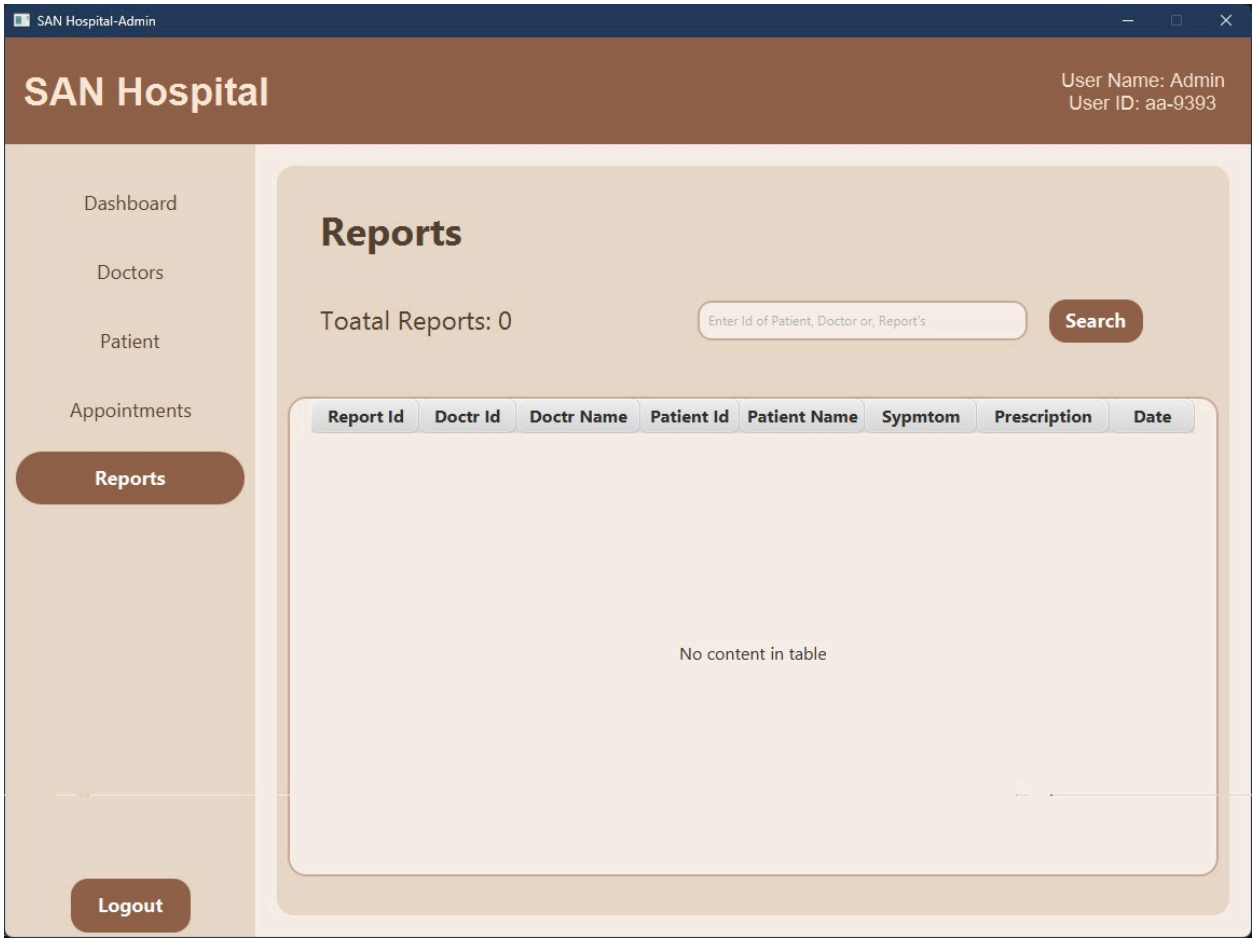
(When Admin click Add Patient, it tells Admin to add a Patient by entering all the Patient information)



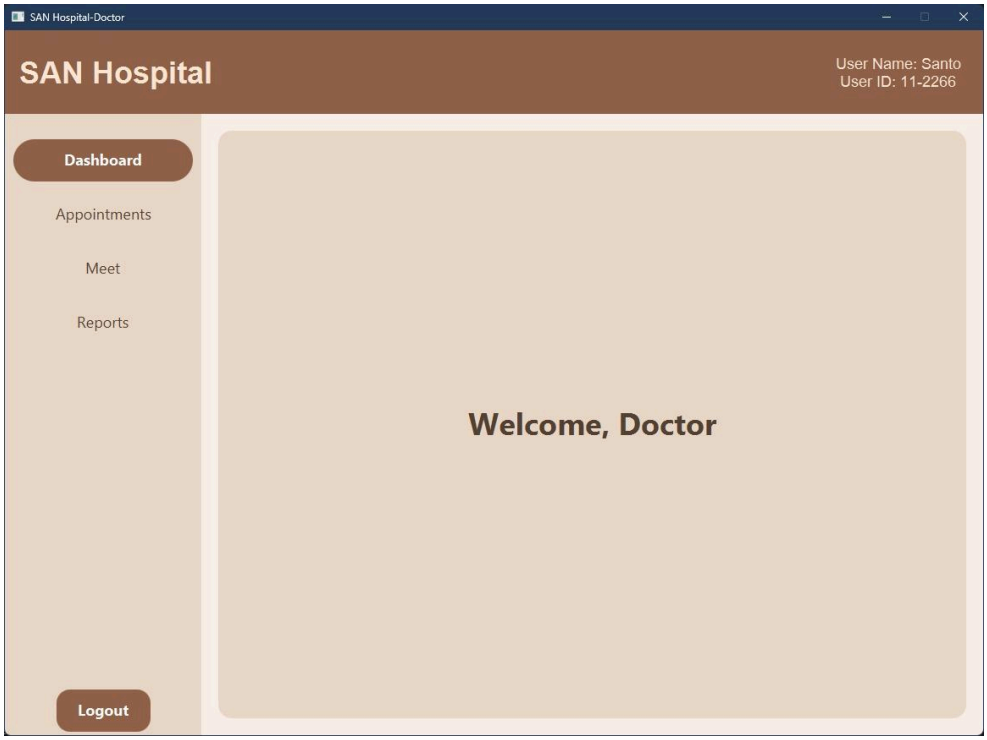
(This is the Appointment List page where the Admin can view all appointments)



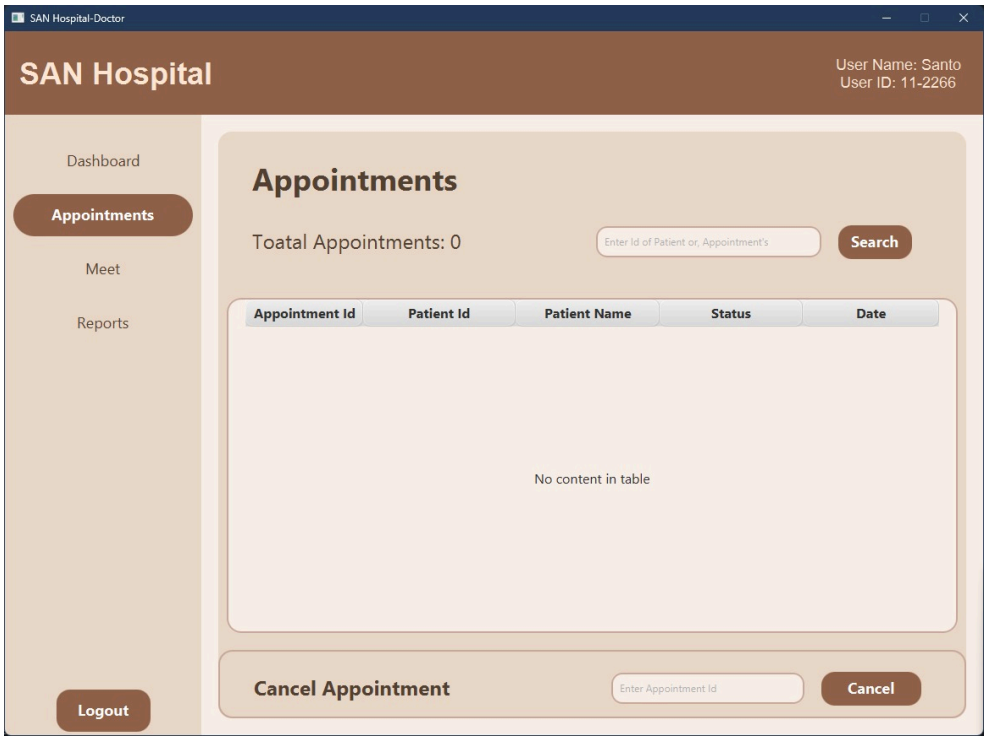
(When Admin click Create Appointment, it tells Admin to Enter Doctor Id , Patient Id)



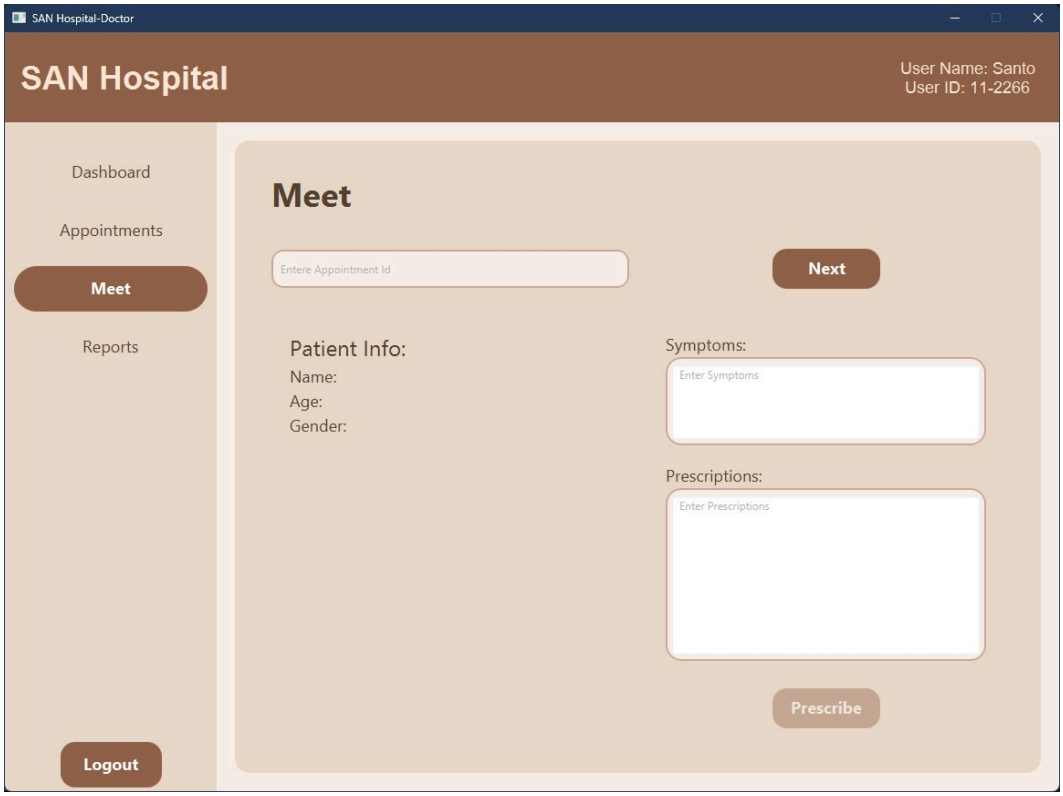
(This is the Report List page where the Admin can view all reports)



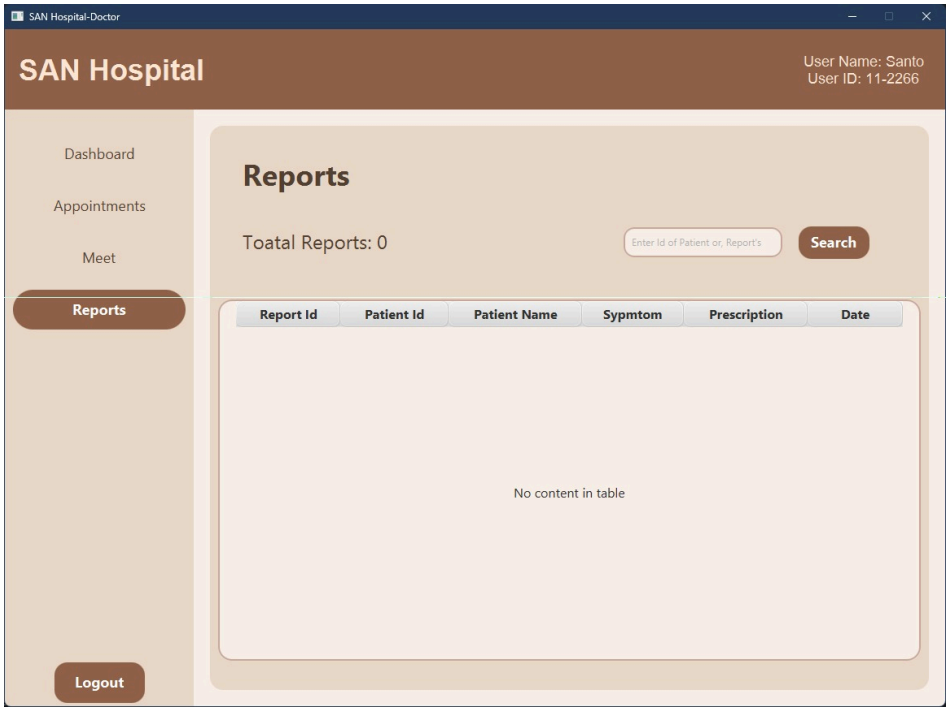
(This is the Dashboard for Doctor Page)



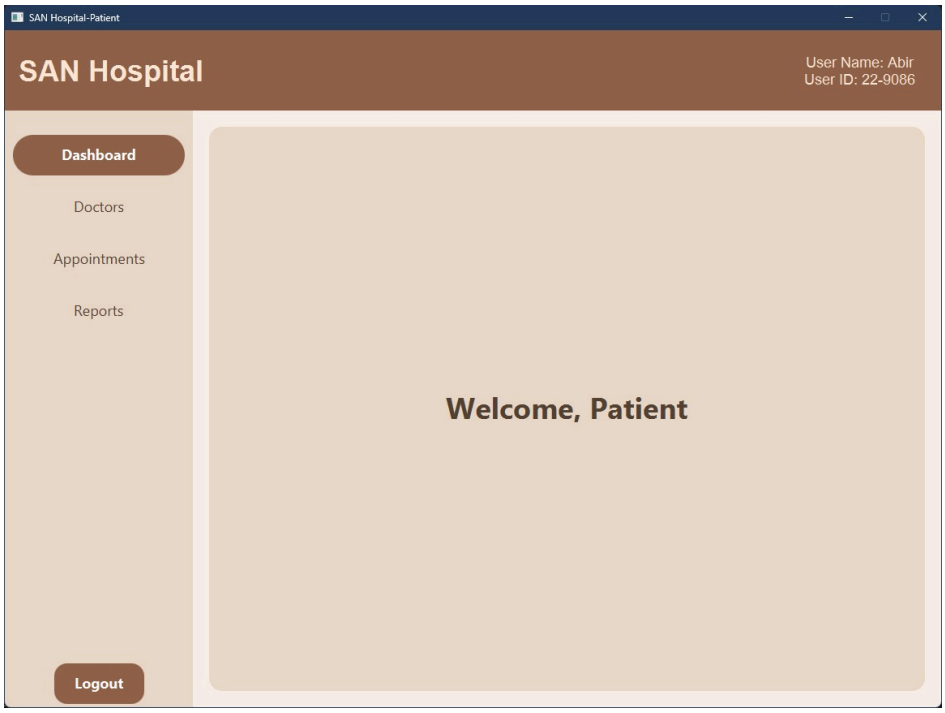
(This is the Appointment List page where the Doctor can view all appointments)



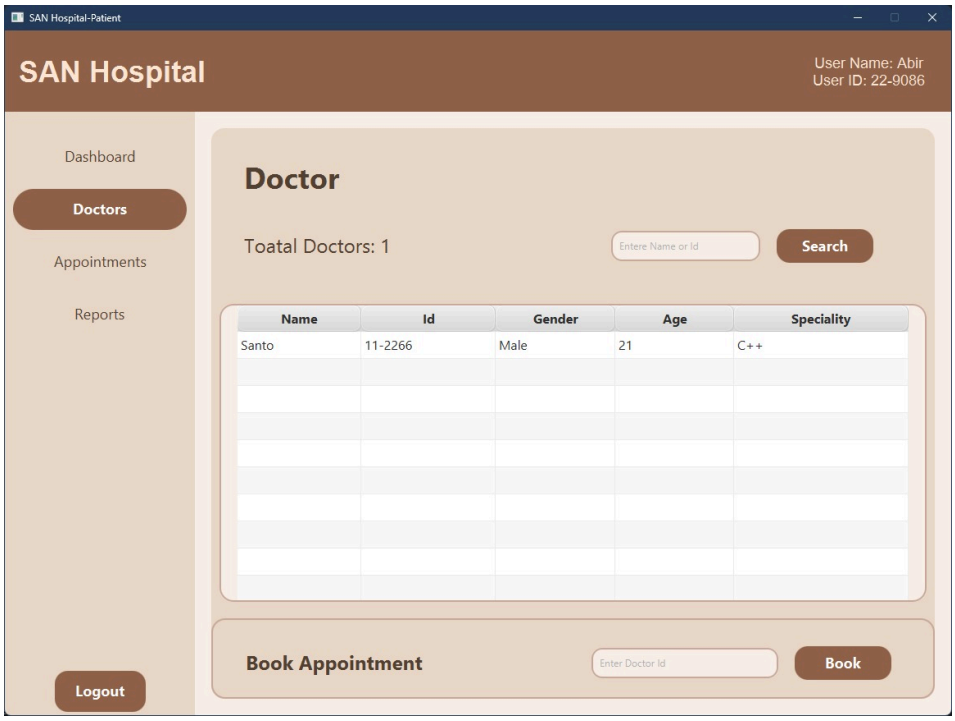
(This is the Prescription page where the Doctor can prescribe medicines)



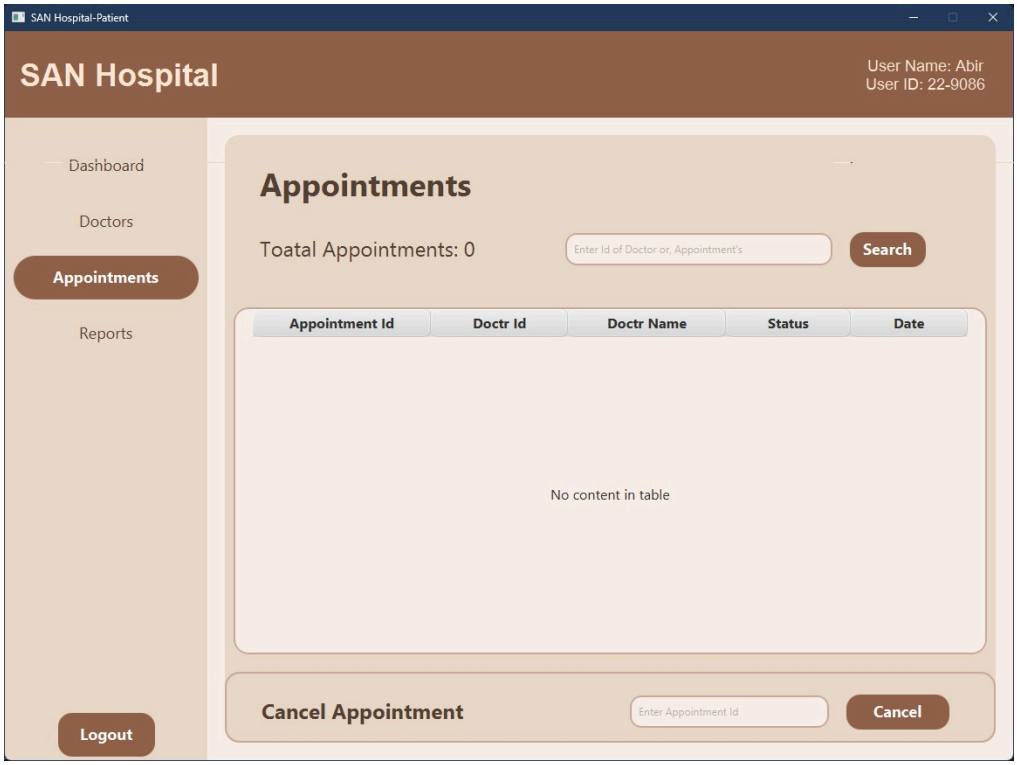
(This is the Report List page where the Doctor can view all reports)



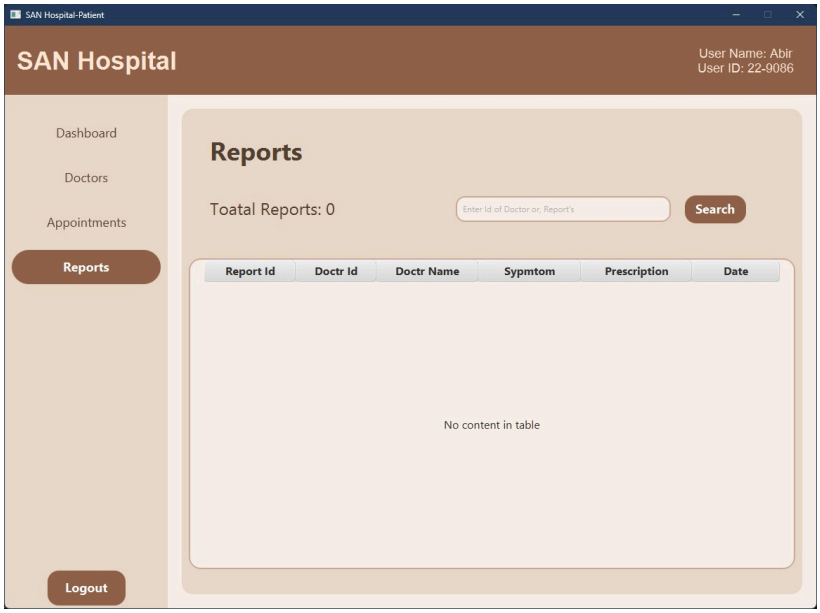
(This is the Dashboard for Patient Page)



(This is a Doctor list of Patient page where Patient can see the Doctor Information)



(This is the Appointment List page where the Patient can view all appointments)



(This is the Report List page where the Patient can view all reports)

7. Member Contribution

A table containing the planned tasks and completed tasks of each member. Every member is responsible for working on both 1) the FXML files and 2) the corresponding controller classes, ensuring that the code uses one or more methods from the backend project.

Member ID: Name	***Planned Tasks***	***Completed Tasks***
24101049- Santo	Admin Boot page, Admin(Doctor ,Appointment, Patient, Report) page, Patient Register page, Dashboard Page, Link pages	Admin Boot page, Admin(Doctor ,Appointment, Patient, Report) page, Patient Register page, Dashboard Page, Link pages
24101035- Noshin	Design, Select User Type page code, Patient(Appointment,Doctor, Report) page	Design, Select User Type page code,Patient(Appointment,Doctor, Report) page
24101033- Samiha	Design, Front page code, Doctor (Appointment,meet, Report) page	Design, Front page code, Doctor (Appointment,meet, Report) page
24101051- Abir	Login Page code, Doctor Dashboard, Patient Dashboard	Login Page code, Doctor Dashboard, Patient Dashboard

8. Conclusion

8.1 : Challenges Faced during Project

During the development of this project, we faced several technical and organizational challenges. One of the initial difficulties was setting up JavaFX properly. Due to version mismatches and missing configurations, the environment took multiple attempts to get working. We also encountered frequent coding errors, especially related to class linking, package structure, and method access. At times, connecting different parts of the project—such as appointments, patient records, and user classes—created confusion and required careful debugging.

Another challenge arose during the GUI design phase. At first, it was difficult to decide how to structure the interface and select the appropriate components. Understanding how the GUI would interact with the backend also required time, and several trial-and-error attempts were needed to finalize a workable layout.

As this was a group project, coordinating the workload also presented challenges. Merging each member's code occasionally caused conflicts, and ensuring that all modules worked together smoothly took extra time. Although these issues slowed down our progress, overcoming them strengthened our understanding of Java, improved our debugging techniques, and enhanced our ability to work collaboratively.

8.2 Lesson Learned (Both Technical and Management aspects) :

Technical Lessons

Through the development of this project, our group gained a clearer understanding of how Object-Oriented Programming concepts can be applied in a practical system. We learned how inheritance helps eliminate redundancy, how encapsulation maintains data integrity, and how abstraction simplifies complex structures by focusing on essential features. Additionally, working with multiple classes, packages, and custom exceptions strengthened our ability to design organized, scalable code. Overall, the project enhanced our technical confidence in building a structured Java application.

Management Lessons

As this was a group assignment, we also developed important teamwork and project management skills. We learned to divide responsibilities effectively, communicate regularly, and ensure consistency across different parts of the system. Coordinating the development of interconnected modules—such as doctors, patients, appointments, and records—taught us the value of planning and version control. The process also highlighted the importance of reviewing each member's work and integrating the components systematically. Collectively, the project improved our collaboration, time management, and problem-solving abilities.